

- 必須ではありませんが、ウィジェットの最適化や再利用性を高めるために推奨される。

d. Java との比較

- Dart も Java も `extends` を使ってサブクラスを作る。
- Dart の方がシンプルで、Flutter のウィジェット設計に最適化されている。
- Java では通常、UI は別ファイルで定義されるが、Flutter ではウィジェットクラス内に直接書く。

```
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

ステップ5：@override を使ったメソッドのオーバーライド

a. Flutter における @override とは？

- @override は、子クラスのメソッドが親クラスのメソッドを置き換えるときに使う。
- Flutter では、@override は StatelessWidget や StatefulWidget から継承された build() によく使用される。
- 親クラスのメソッドを正しくオーバーライドしていることを保証する。
- build() メソッドをオーバーライドする例：

```
@override
Widget build(BuildContext context) {
```

- 説明:
 - ◇ @override → この build() メソッドが親クラスの build() を置き換えていることを Flutter に伝える。
 - ◇ build(BuildContext context) → UI の表示方法を定義する。
 - ◇ @override がなくても Dart は動作するが、コードの明確性を向上させ、ミスを防ぐのに役立つ。

b. なぜ @override が重要なのか？

- メソッドが正しくオーバーライドされていることを保証する。
- コードを読みやすくする → このメソッドが親クラスのメソッドを置き換えていることを明示的に示す。
- 意図しないミスを防ぐ → @override がないと、メソッド名を間違えたときに Dart がエラーを検出しない可能性がある。

c. 比較：Java と Flutter における @override

- Java も Flutter (Dart) も @override を使って、メソッドのオーバーライドを明示する。

- メソッドのシグネチャが一致しない場合のミスを防ぐ.
- Java では明示的なメソッドシグネチャが必要だが,Dart はより柔軟.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- 説明:
 - ◇ `@Override` → Child の `build()` が Parent の `build()` を正しくオーバーライドしていることを保証する.
 - ◇ メソッド名が間違っていた場合,Java はエラーを出す.
- 主な違い:

機能	Java	Flutter
<code>@override</code> の用途	OOP メソッドのオーバーライド	Flutter ウィジェットのオーバーライド
エラー防止	誤ったメソッドシグネチャを防ぐ	タイプミスの検出を助ける
柔軟性	厳格なメソッド定義	より柔軟だが, <code>@override</code> で明確

ステップ6：build() メソッドとBuildContextの探索

a. build()メソッドとは？

- `build()` メソッドは,ウィジェットが画面にどのように表示されるかを定義する.
- “ウィジェットツリー” (UIを構成するウィジェットの階層構造) を返す.
- UIが更新される必要があるときに,`build()` メソッドが再び呼び出される.

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text('Hello, Flutter!'),
                ),
            ),
        );
    }
}
```